Cyfrowe Archiwum Społecznościowe

Dokumentacja systemu

Autor: Jakub Socała

Wersja:	1.0
Data:	9 kwietnia 2026
Backend:	ASP.NET Core 8, PostgreSQL
Frontend:	Vue 3, Vite, TypeScript

Dokument opisuje architekturę systemu, model danych, strukturę metadanych oraz analizę dostępności zgodną ze standardem WCAG 2.1 AA.

Spis treści

1 Opis projektu **3**
1.1 Główne założenia . . . 3
1.2 Role w systemie . . . 4

2 Uzasadnienie przyjętych rozwiązań technicznych **4**
2.1 Podział na backend i frontend (architektura SPA) . . . 4
2.2 Technologie na backendzie . . . 4
2.3 Technologie na frontendzie . . . 4
2.4 PostgreSQL jako baza danych . . . 5
2.5 Autoryzacja JWT + OAuth (bez tradycyjnego hasła) . . . 5
2.6 Zmienna precyzja dat . . . 5
2.7 Trzy motywy wizualne . . . 6

3 Architektura systemu **7**
3.1 Diagram warstw . . . 7
3.2 Struktura backendu . . . 7
3.3 Struktura frontendu . . . 8
3.4 Przepływ autoryzacji OAuth . . . 9

4 Integracja frontend–backend, walidacja i obsługa błędów **9**
4.1 Komunikacja HTTP . . . 9
4.2 Zarządzanie stanem (Pinia) . . . 10
4.3 Route guards (autoryzacja nawigacji) . . . 10
4.4 Walidacja danych . . . 11
4.4.1 Frontend (przed wysłaniem żądania) . . . 11
4.4.2 Backend (po otrzymaniu żądania) . . . 11
4.5 Obsługa błędów . . . 11

5 Architektura informacji **12**
5.1 Hierarchiczna struktura kategorii . . . 12
5.2 Mapa strony . . . 13
5.3 Schemat organizacji treści . . . 15
5.4 Taksonomia i tagowanie . . . 15
5.5 Ścieżki użytkownika . . . 16
5.5.1 Przeglądający (bez konta) . . . 16
5.5.2 Twórca (zalogowany) . . . 16
5.5.3 Administrator . . . 16

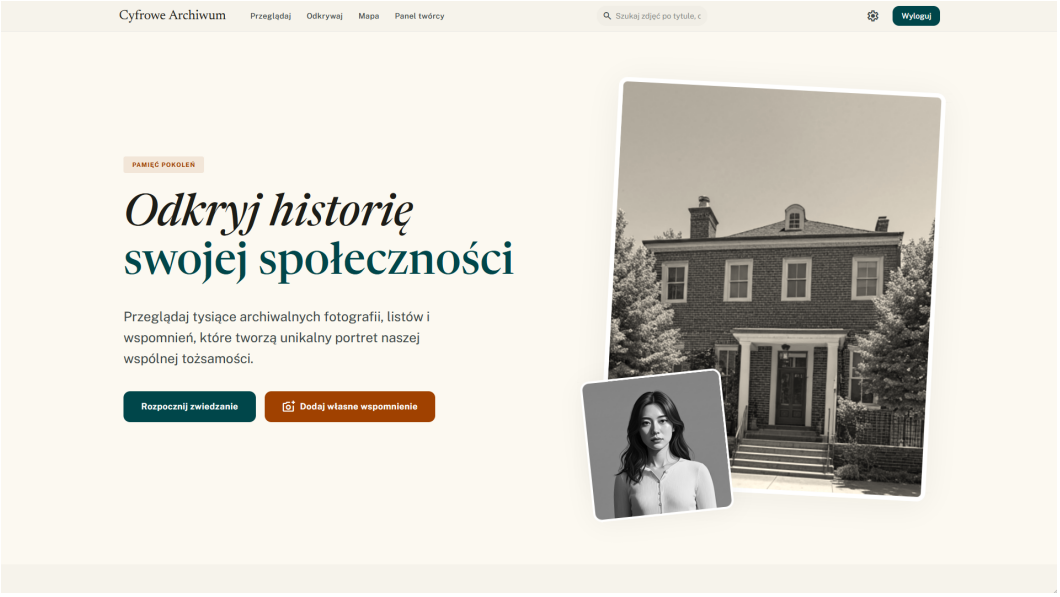
6 Model danych **17**
6.1 Diagram encji (ERD) . . . 17
6.2 Tabela: User . . . 18
6.3 Tabela: Photo . . . 19
6.4 Tabela: Category . . . 20
6.5 Tabela: Tag . . . 20
6.6 Tabela: Comment . . . 20

7 Struktura metadanych **20**

7.1 Klasyfikacja metadanych . . . 20
7.1.1 Grupa 1: Identyfikacyjne (wymagane) . . . 21
7.1.2 Grupa 2: Lokalizacyjne . . . 21
7.1.3 Grupa 3: Opisowe . . . 21
7.1.4 Grupa 4: Techniczne / archiwistyczne . . . 21
7.2 Uzasadnienie struktury metadanych . . . 22
7.3 Parametry wyszukiwania a pola bazy danych . . . 22

8 Dokumentacja API 22
8.1 Legenda dostępu . . . 22
8.2 Autentykacja (`/api/auth`) . . . 23
8.3 Zdjęcia (`/api/photos`) . . . 23
8.3.1 Parametry `GET /api/photos` . . . 24
8.3.2 Body `POST /api/photos` (`multipart/form-data`) . . . 24
8.3.3 Body `PUT /api/photos/{id}` (`application/json`) . . . 24
8.4 Kategorie (`/api/categories`) . . . 25
8.5 Komentarze . . . 25
8.6 Administracja (`/api/admin`) . . . 25
8.7 Przykładowe odpowiedzi . . . 26
8.7.1 `GET /api/photos` — odpowiedź stronicowana . . . 26
8.7.2 `POST /api/auth/google` — odpowiedź po zalogowaniu . . . 27
8.8 Autoryzacja — szczegóły tokenu JWT . . . 27
8.9 Zestawienie wszystkich endpointów . . . 27

9 Analiza dostępności informacji (WCAG 2.1 AA) 28
9.1 Przegląd . . . 28
9.2 Postrzegalność (Perceivable) . . . 28
9.2.1 1.1.1 — Treść nietekstowa (poziom A) . . . 28
9.2.2 1.3.1 — Informacje i relacje (poziom A) . . . 28
9.2.3 1.4.3 — Kontrast (poziom AA) . . . 29
9.2.4 1.4.4 — Zmiana rozmiaru tekstu (poziom AA) . . . 29
9.2.5 1.4.13 — Zawartość przy najechaniu / fokusie (poziom AA) . . . 29
9.3 Funkcjonalność (Operable) . . . 29
9.3.1 2.1.1 — Klawiatura (poziom A) . . . 29
9.3.2 2.4.1 — Pomijanie bloków (poziom A) . . . 29
9.3.3 2.4.4 — Cel linku (poziom A) . . . 30
9.3.4 2.4.6 — Nagłówki i etykiety (poziom AA) . . . 30
9.3.5 2.3.3 — Animacja z interakcji (poziom AAA — zrealizowane) . . . 30
9.4 Zrozumiałość (Understandable) . . . 30
9.4.1 3.1.1 — Język strony (poziom A) . . . 30
9.4.2 3.3.1 — Identyfikacja błędu (poziom A) . . . 30
9.4.3 3.3.2 — Etykiety lub instrukcje (poziom A) . . . 30
9.5 Solidność (Robust) . . . 30
9.5.1 4.1.2 — Nazwa, rola, wartość (poziom A) . . . 30
9.5.2 4.1.3 — Komunikaty o stanie (poziom AA) . . . 31
9.6 Zestawienie kryteriów WCAG 2.1 AA . . . 31
9.7 Dostosowanie wyglądu . . . 32



Rysunek 1: Widok strony głównej

1 Opis projektu

Cyfrowe Archiwum Społecznościowe to aplikacja internetowa umożliwiająca lokalnym mieszkańcom i pasjonatom historii gromadzenie, opisywanie i udostępnianie zdjęć dokumentujących historię i ewolucję danego obszaru geograficznego.

Link do repozytorium: https://github.com/JakubSocalaPW/Aitsi

1.1 Główne założenia

- **Dostępność** — materiały historyczne mają charakter społecznościowy; interfejs musi być prosty i intuicyjny, by mogli z niego korzystać również starsi użytkownicy.
- **Wyszukiwanie kontekstualne** — możliwość wyszukiwania według lokalizacji geograficznej, okresu historycznego oraz hierarchicznej struktury obszarów.
- **Trójpoziomowy model ról** — zróżnicowane uprawnienia dla Przeglądającego, Twórcy i Administratora.
- **Publiczne API** — otwarte wyszukiwanie bez logowania, prywatne API dla zarządzania treścią.

1.2 Role w systemie

Tabela 1: Role użytkowników i ich uprawnienia

Rola	Uprawnienia
Przeglądający	Przeglądanie zdjęć, wyszukiwanie po frazie / kategorii / dacie / tagu
Twórca	Wszystkie uprawnienia Przeglądającego + logowanie przez OAuth, dodawanie / edycja / usuwanie własnych zdjęć, komentowanie
Administrator	Wszystkie uprawnienia Twórcy + zarządzanie kategoriami, edycja metadanych dowolnego zdjęcia, usuwanie zdjęć, blokowanie / odblokowywanie użytkowników

2 Uzasadnienie przyjętych rozwiązań technicznych

2.1 Podział na backend i frontend (architektura SPA)

System zbudowano w architekturze Single Page Application z oddzielnym backendem REST API. Podejście to umożliwia:

- niezależne skalowanie warstwy prezentacji i warstwy danych,
- udostępnienie tego samego API innym klientom,
- usprawnienie procesu tworzenia oprogramowania.

2.2 Technologie na backendzie

- **.NET Core 8** — framework w C# do tworzenia API.
- **Entity Framework Core** — ORM, migracje, silnie typowane zapytania LINQ, automatyczna obsługa relacji wiele-do-wielu (`Photo` ↔ `Tag`).
- **JWT Bearer** — bezstanowa autoryzacja kompatybilna z zewnętrznymi dostawcami tożsamości (Google, Facebook).
- **PostgreSQL** — relacyjna baza danych z dojrzałym wsparciem w .NET
- **Swagger** — automatyczna dokumentacja API dostępna pod `/swagger`.

2.3 Technologie na frontendzie

- **Vue 3** — framework z wysoką wydajnością i małym rozmiarem, wykorzystujący w projekcie Typescript
- **Composition API** — dobra organizacja logiki przez composables (`usePhotoSearch`, `useAnnouncer`, `usePagination`)
- **Pinia** — lekki store z pełnym wsparciem TypeScript; jeden store na domenę (photos, categories, auth, theme, fontsize, toast).

- **Vue Router** — nawigacja zapewniająca blokadę dostępu do ścieżek
- **ky** — minimalistyczny klient HTTP oparty na Fetch API.

2.4 PostgreSQL jako baza danych

Wybór relacyjnej bazy danych uzasadniony przez:

- złożone relacje (`User` → `Photo` → `Category` → `Tag` → `Comment`),
- konieczność zachowania integralności referencyjnej (DELETE RESTRICT na kategoriach, CASCADE na komentarzach),
- pełnotekstowe wyszukiwanie po polach tekstowych bez dodatkowego silnika,
- hierarchiczna struktura kategorii jako samoreferencja w jednej tabeli.

2.5 Autoryzacja JWT + OAuth (bez tradycyjnego hasła)

Zrezygnowano z rejestracji e-mail/hasło na rzecz wyłącznie OAuth (Google, Facebook). Uzasadnienie:

- eliminacja konieczności przechowywania haseł i zarządzania ich bezpieczeństwem,
- niższy próg wejścia dla użytkownika — jedno kliknięcie zamiast formularza,
- weryfikacja adresu e-mail leży po stronie dostawcy tożsamości.

Przy pierwszym logowaniu przez OAuth system automatycznie tworzy konto z rolą `Creator`.

2.6 Zmienna precyzja dat

Materiały historyczne często posiadają nieprecyzyjne daty. Pole `DatePrecision` przyjmuje wartości `Year`, `Month`, `Day`:

- data przechowywana jako string w formacie `YYYY`, `YYYY-MM` lub `YYYY-MM-DD`,
- filtrowanie zakresu dat inteligentnie dopełnia datę końcową (np. `"1960"` → `"1960-12-31"`),
- komponent `DatePrecisionPicker` pozwala wybrać granularność.

2.7 Trzy motywy wizualne

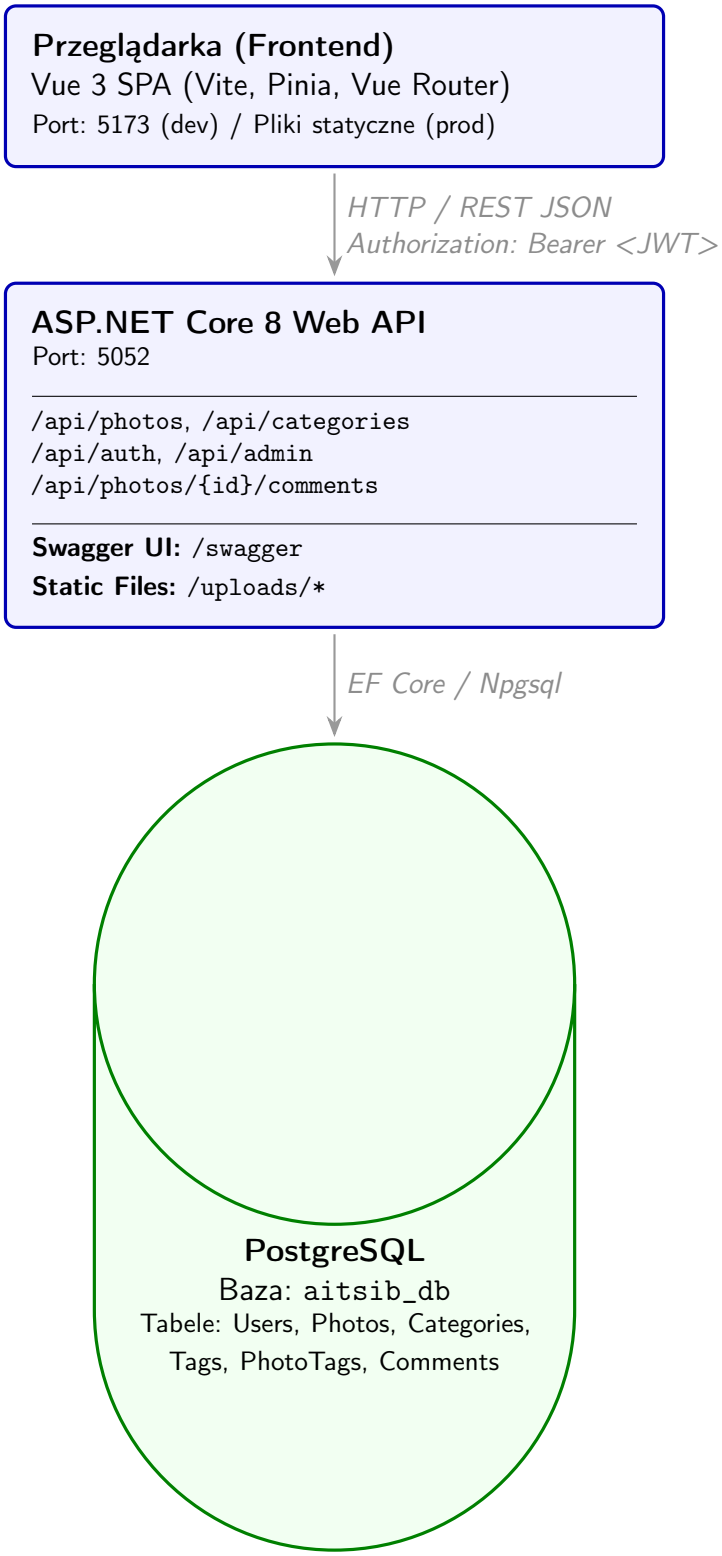
Tabela 2: Motywy wizualne systemu

Motyw	Trigger	Przeznaczenie
`light`	Domyślny lub `prefers-color-scheme: light`	Standardowe użytkowanie
`dark`	`prefers-color-scheme: dark` lub ręcznie	Komfort w słabym oświetleniu
`high-contrast`	`prefers-contrast: more` lub ręcznie	Osoby słabowidzące

Wybór trzymany w `localStorage` (klucz `cas_theme`). Zmiana następuje natychmiastowo przez atrybut `data-theme` na `<html>` bez przeładowania strony.

3 Architektura systemu

3.1 Diagram warstw



3.2 Struktura backendu

Listing 1: Drzewo katalogów backendu

```
backend/
|-- Controllers/
|   |-- AuthController.cs        logowanie OAuth, /auth/me
|   |-- PhotosController.cs      CRUD zdjec, wyszukiwanie,
   upload
|   |-- CategoriesController.cs  zarzadzanie hierarchia
   kategorii
|   |-- CommentsController.cs    komentarze do zdjec
|   `-- AdminController.cs       panel admina (uzytkownicy,
   statystyki)
|-- Models/
|   |-- User.cs                  enum UserRole, pola OAuth
|   |-- Photo.cs                 enum DatePrecision, metadane
|   |-- Category.cs              self-referencing tree
|   |-- Tag.cs                   tagi (relacja M:N z Photo)
|   `-- Comment.cs
|-- Data/
|   `-- AppDbContext.cs          konfiguracja EF, relacje, seed
   admina
|-- Migrations/                  historia schematu bazy
|-- Program.cs                   konfiguracja DI, JWT, CORS,
   Swagger
`-- appsettings.Development.json connection string, klucz JWT
```

3.3 Struktura frontendu

Listing 2: Drzewo katalogów frontendu (src/)

```
frontend/src/
|-- pages/             widoki (1 plik = 1 strona/trasa)
|-- components/
|   |-- layout/        AppHeader, AppNav, AppFooter, MobileMenu
|   |-- photos/        PhotoCard, PhotoGrid, PhotoFilters,
   PhotoSearchBar
|   |-- forms/         PhotoUploadForm, ImageDropzone,
   DatePrecisionPicker
|   |-- categories/    CategoryBreadcrumb, CategoryForm
|   |-- admin/         AdminPhotoTable, UserTable,
   BlockUserDialog
|   `-- common/        AppPagination, AppSpinner, AppToast,
   SettingsDropdown
|-- stores/            Pinia stores (1 store = 1 domena)
|-- composables/
|   |-- usePhotoSearch.ts   synchronizacja filtrow z URL
|   |-- useAnnouncer.ts     ARIA live region dla czytnikow
   ekranu
|   |-- usePagination.ts
|   `-- useBreakpoint.ts
|-- api/               klienci HTTP (1 plik = 1 zasob)
|-- router/            trasy + navigation guards
|-- types/             interfejsy TypeScript (DTOs)
`-- i18n/              tlumaczenia (jezyk polski)
```

3.4 Przepływ autoryzacji OAuth

Listing 3: Sekwencja logowania przez OAuth

```
Uzytkownik klika "Zaloguj przez Google"
        |
        v
Frontend pobiera token od Google (client-side OAuth flow)
        |
        v
POST /api/auth/google { token: "<google_id_token>" }
        |
        v
Backend weryfikuje token u Google
-> pobiera email, displayName, avatarUrl
        |
        +-- Nowy uzytkownik --> tworzy konto z rola Creator
        `-- Istniejacy     --> pobiera dane z bazy
        |
        v
Backend zwraca wlasny JWT
  payload: { userId, email, role }
        |
        v
Frontend zapisuje JWT w localStorage (auth.store)
        |
        v
Kazde kolejne zadanie:
  Authorization: Bearer <JWT>
```

4 Integracja frontend–backend, walidacja i obsługa błędów

4.1 Komunikacja HTTP

Frontend komunikuje się z backendem przez bibliotekę `ky` — minimalistyczny klient HTTP oparty na Fetch API. Centralna instancja klienta tworzona jest w `api/client.ts` i automatycznie dołącza nagłówek autoryzacyjny do każdego żądania:

```
export const apiClient = ky.create({
  prefixUrl: import.meta.env.VITE_API_URL ?? 'http://localhost
   :5052/api',
  hooks: {
    beforeRequest: [
      (request) => {
        const token = localStorage.getItem('cas_token')
        if (token) request.headers.set('Authorization', `Bearer
   ${token}`)
      }
    ]
```

```
  }
})
```

Listing 4: Konfiguracja centralnego klienta HTTP (api/client.ts)

Każdy zasób API ma własny moduł (`photos.api.ts`, `categories.api.ts` itd.) eksportujący funkcje z silnym typowaniem TypeScript — odpowiedniki DTO z backendu.

4.2 Zarządzanie stanem (Pinia)

Dane pobrane z API trafiają do Pinia stores, skąd są reaktywnie dostępne dla komponentów:

Listing 5: Przepływ danych: API -> Store -> Komponent

```
photosApi.search(params)   ->  photos.store.ts   ->  BrowsePage.
   vue / PhotoGrid.vue
adminApi.getUsers()        ->  (lokalny ref)     ->
   AdminUsersPage.vue
authApi.getMe()            ->  auth.store.ts     ->  AppHeader.
   vue / guards.ts
```

4.3 Route guards (autoryzacja nawigacji)

Plik `router/guards.ts` sprawdza meta-pola trasy przed każdą nawigacją:

```
requiresAuth: true   -> uzytkownik musi byc zalogowany (JWT w
   localStorage)
requiresAdmin: true  -> rola uzytkownika musi byc 'admin'
guestOnly: true      -> zalogowany uzytkownik przekierowany do /
   przegladaj
```

Przy błędzie 401 z API użytkownik jest automatycznie wylogowywany i przekierowany na stronę logowania.

4.4 Walidacja danych

4.4.1 Frontend (przed wysłaniem żądania)

Tabela 3: Reguły walidacji po stronie frontendu

Pole	Reguła	Komponent
Plik zdjęcia	Wymagany; typy: jpg, png, webp; max 20 MB	`ImageDropzone.vue`
Tytuł	Wymagany; max 200 znaków	`PhotoUploadForm.vue`
Kategoria	Wymagana (select)	`CategorySelect.vue`
Data	Format `YYYY/YYYY-MM/YYYY-MM-DD`; zakres 1800-bieżący rok	`DatePrecisionPicker.vue`
Opis	Max 2000 znaków	`maxlength` na textarea
Cytat	Max 1000 znaków	`maxlength` na textarea
Powód blokady	Wymagany (dialog blokady)	`BlockUserDialog.vue`

4.4.2 Backend (po otrzymaniu żądania)

- Plik: sprawdzany przez `IFormFile` — brak pliku zwraca `400 Bad Request`.
- Token OAuth: Google — weryfikacja przez `googleapis.com/tokeninfo`; Facebook — weryfikacja przez `graph.facebook.com`.
- Autoryzacja: `[Authorize]` i `[Authorize(Roles = "admin")]` na metodach kontrolera.
- Własność zasobu: przy `PUT`/`DELETE` na zdjęciu sprawdzany `AuthorId == currentUserId` lub rola admin.

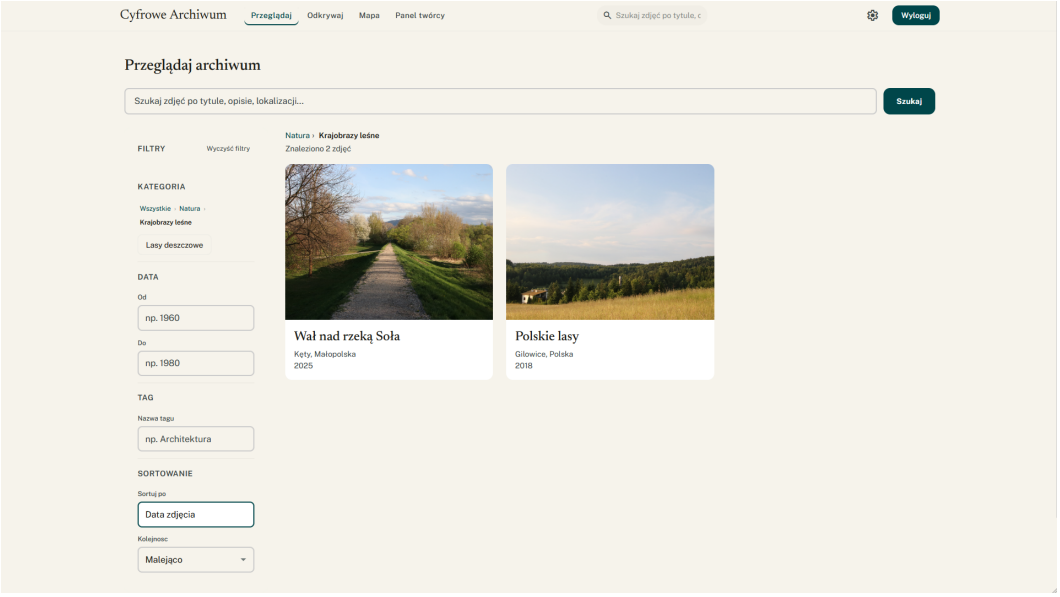
4.5 Obsługa błędów

Listing 6: Hierarchia obsługi błędów

```
Blad sieciowy / timeout
    `-- ky rzuca HTTPError lub TypeError
            `-- catch w bloku try/catch w api/*.ts lub
               komponencie
                   `-- toastStore.show('Wystapil blad', 'error')
                           `-- AppToast.vue wyswietla komunikat (
                              aria-live="assertive")

Blad 401 Unauthorized
    `-- uzytkownik wylogowany, redirect -> /logowanie

Blad 403 Forbidden
    `-- komunikat toast "Brak uprawnien"
```



Rysunek 2: Filtrowanie wykorzystujące kategorie

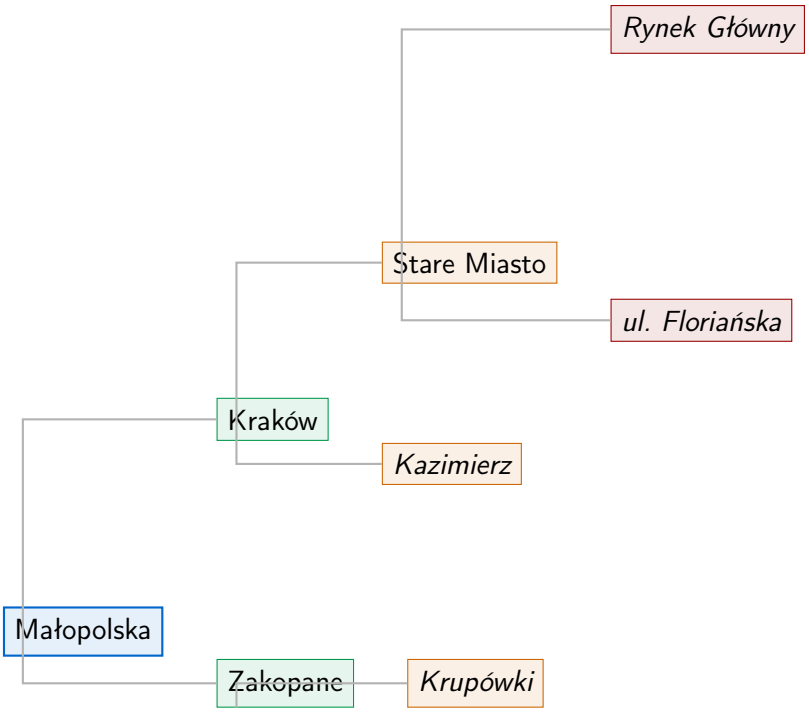
```
Blad 404 Not Found
    `-- redirect -> NotFoundPage.vue
```

Przy próbie logowania zablokowanego konta backend zwraca JSON z polem `message` (np. `"Konto zostało zablokowane. Powód: spam"`). Frontend odczytuje tę wiadomość i wyświetla ją użytkownikowi zamiast generycznego komunikatu o błędzie.

5 Architektura informacji

5.1 Hierarchiczna struktura kategorii

Główną osią organizacji treści jest drzewo kategorii geograficzno-tematycznych. Każda kategoria może mieć dowolną liczbę podkategorii, a głębokość drzewa jest nieograniczona.



Relacja w bazie danych: samoreferencja tabeli `Categories` przez pole `ParentId`. Usunięcie kategorii jest zablokowane (`DeleteBehavior.Restrict`) gdy posiada podkategorie lub przypisane zdjęcia.

5.2 Mapa strony

Poniżej pełna mapa wszystkich tras aplikacji wraz z poziomem dostępu:

/ (Strona główna)

- `/przegladaj` — Archiwum (Publiczna)
  - `/:id` — Wyniki dla kategorii
- `/zdjecie/:id` — Szczegóły zdjęcia
- `/mapa` — Mapa interaktywna
- `/kroniki` — Kroniki społeczności
- `/logowanie` — Logowanie OAuth (Goście)

- **Strefa Twórcy**
  - `/dodaj-zdjecie` — Upload zdjęcia
  - `/moje-zdjecia` — Moje materiały
    - `/edytuj-zdjecie/:id` — Edycja zdjęcia
  - `/panel-tworcy` — Dashboard twórcy
- **Strefa Administratora**
  - `/admin` — Panel główny
    - `/moderacja` — Kolejka nowych zdjęć
    - `/zdjecia` — Wszystkie zdjęcia
    - `/uzytkownicy` — Zarządzanie użytkownikami
    - `/kategorie` — Zarządzanie kategoriami

Uzasadnienie przyjętej struktury URL:

- Hierarchia `/przegladaj` → `/przegladaj/:id` rozwiązuje problem orientacji — użytkownik wie, że jest „wewnątrz"archiwum, a kategoria jest podkontekstem.
- Prefix `/admin/*` izoluje strefy administracyjne i umożliwia jednolitą ochronę przez route guard (`requiresAdmin`).
- Polskie URL-e (`/przegladaj`, `/zdjecie`, `/moje-zdjecia`) są spójne z językiem interfejsu i poprawiają czytelność.

5.3 Schemat organizacji treści

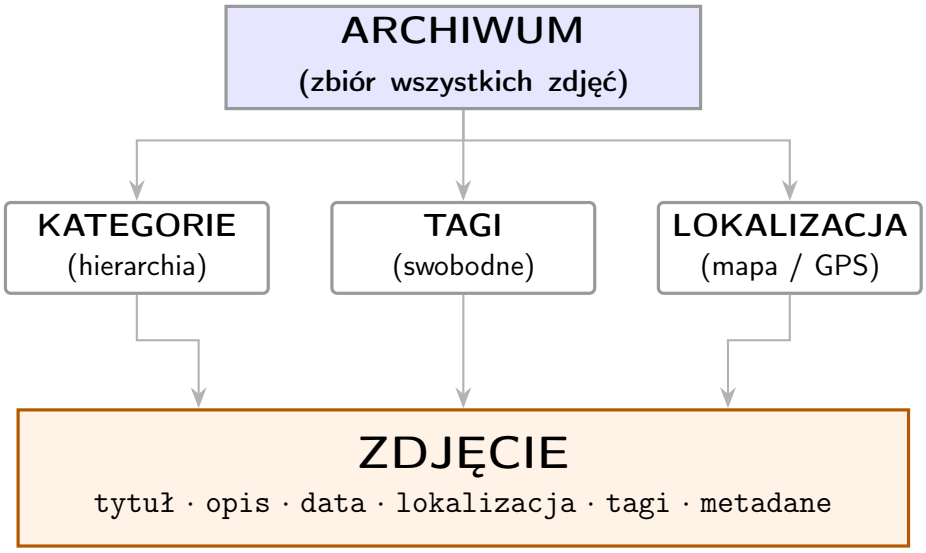


Tabela 4: Problemy użytkownika a rozwiązania struktury informacji

Problem	**Rozwiązanie**
Użytkownik nie zna nazwy miejsca	Filtrowanie po mapie i współrzędnych GPS
Zdjęcie ma tylko przybliżoną datę	Zmienna precyzja daty (rok / miesiąc / dzień)
To samo zdjęcie pasuje do wielu tematów	Tagi swobodne niezależne od hierarchii kategorii
Duże zbiory trudne do przeglądania	Hierarchia kategorii + breadcrumb nawigacyjny
Różne potrzeby różnych użytkowników	Trzy niezależne osie wyszukiwania (kategoria, tag, mapa)

5.4 Taksonomia i tagowanie

System stosuje dwie równoległe metody klasyfikacji:

Tabela 5: Porównanie metod klasyfikacji

Metoda	**Charakter**	**Zarządzanie**	**Przykład**
Kategoria hierarchiczna	Geograficzna / tematyczna	Administrator	Kraków > Stare Miasto
Tagi swobodne	Dowolna klasyfikacja	Twórca	`#architektura`, `#lata60`

Każde zdjęcie musi należeć do dokładnie jednej kategorii i może mieć dowolną liczbę tagów.

5.5 Ścieżki użytkownika

5.5.1 Przeglądający (bez konta)

```
Strona glowna (/)
   |
   +-- Przegladaj archiwum (/przegladaj)
   |     +-- Filtruj po kategorii -> /przegladaj/:categoryId
   |     +-- Filtruj po dacie (od-do)
   |     +-- Filtruj po tagu
   |     +-- Szukaj po frazie
   |     `-- Zdjecie -> /zdjecie/:id
   |
   `-- Mapa (/mapa)
         `-- Pin na mapie -> /zdjecie/:id
```

5.5.2 Twórca (zalogowany)

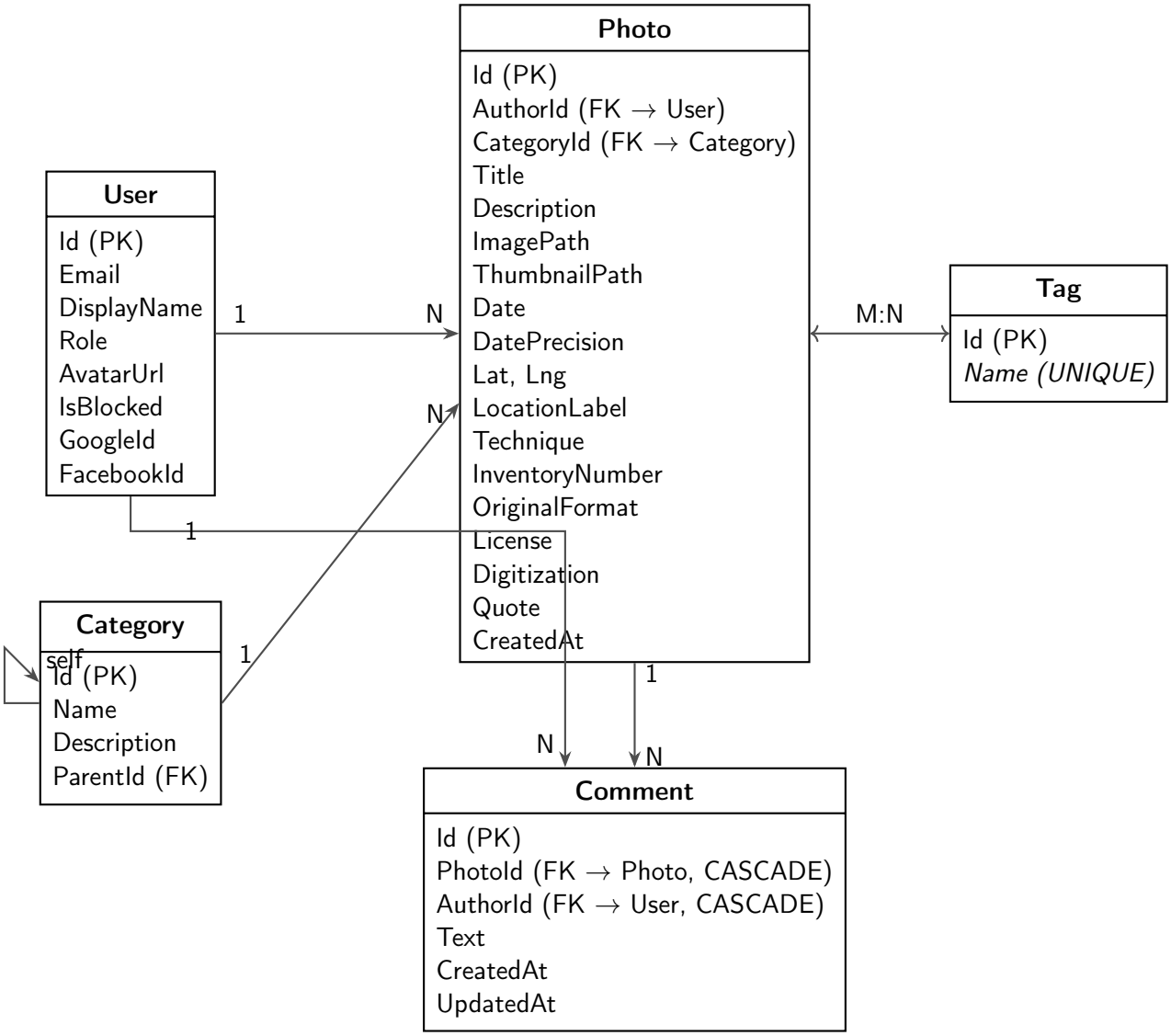
```
Logowanie (/logowanie)
   |
   +-- Moje zdjecia (/moje-zdjecia)
   |     +-- Dodaj zdjecie -> /dodaj-zdjecie
   |     `-- Edytuj -> /edytuj-zdjecie/:id
   |
   `-- Panel tworcy (/panel-tworcy)
```

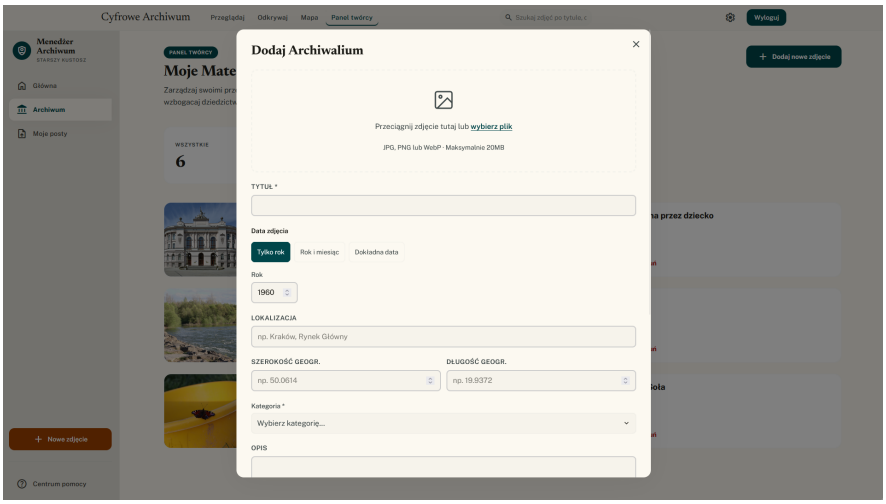
5.5.3 Administrator

```
Panel administracyjny (/admin)
   +-- Moderacja zdjec (/admin/moderacja)
   |     +-- Lista nowych zdjec oczekujacych na akceptacje
   |     +-- Akceptuj -> zdjecie staje sie widoczne publicznie
   |     +-- Podglad -> /zdjecie/:id
   |     `-- Usun -> usuwa zdjecie i plik z dysku
   +-- Wszystkie zdjecia (/admin/zdjecia)
   |     +-- Przegladaj zatwierdzone materialy (paginacja,
      sortowanie)
   |     +-- Edytuj dowolne zdjecie -> /edytuj-zdjecie/:id
   |     `-- Usun dowolne zdjecie
   +-- Zarzadzaj uzytkownikami (/admin/uzytkownicy)
   |     `-- Blokuj / odblokuj uzytkownika (z podaniem powodu)
   `-- Zarzadzaj kategoriami (/admin/kategorie)
         +-- Dodaj kategorie
         +-- Edytuj kategorie
         `-- Usun kategorie (zablokowane jesli ma podkategorie
            lub zdjecia)
```

6 Model danych

6.1 Diagram encji (ERD)



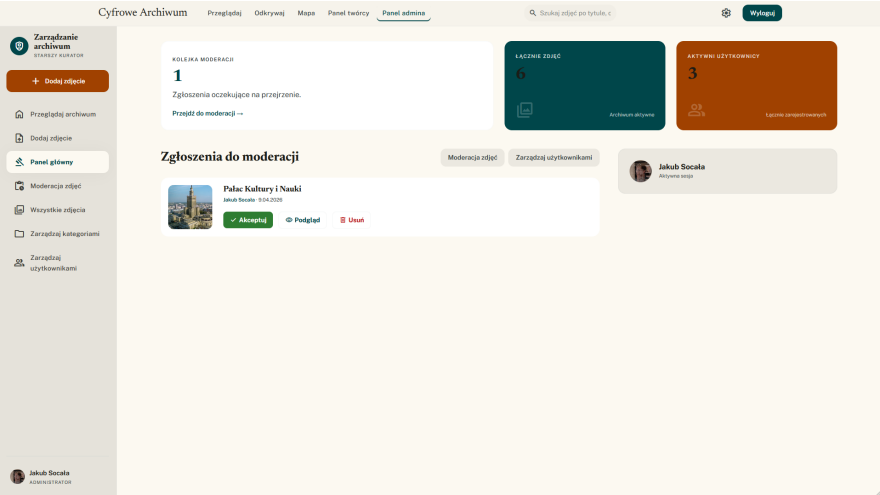


Rysunek 3: Dodawanie zdjęcia w interfejsie

6.2 Tabela: User

Tabela 6: Struktura tabeli User

Kolumna	Typ	Ograniczenia	Opis
`Id`	int	PK, auto	Klucz główny
`Email`	varchar	NOT NULL, UNIQUE	E-mail z OAuth
`DisplayName`	varchar	NOT NULL	Nazwa wyświetlana
`Role`	varchar	NOT NULL	`Viewer` / `Creator` / `Admin`
`AvatarUrl`	varchar	NULL	URL avatara z OAuth
`IsBlocked`	bool	NOT NULL	Czy konto zablokowane
`BlockReason`	varchar	NULL	Powód blokady
`CreatedAt`	timestamp	NOT NULL	Data rejestracji (UTC)
`GoogleId`	varchar	NULL	ID konta Google
`FacebookId`	varchar	NULL	ID konta Facebook



Rysunek 4: Zdjęcia w moderacji

6.3 Tabela: Photo

Tabela 7: Struktura tabeli Photo

Kolumna	Typ	Ogr.	Opis
`Id`	int	PK, auto	Klucz główny
`Title`	varchar	NOT NULL	Tytuł zdjęcia
`Description`	text	NULL	Opis tekstowy
`ImagePath`	varchar	NOT NULL	Ścieżka do pliku
`ThumbnailPath`	varchar	NOT NULL	Ścieżka do miniatury
`IsApproved`	bool	NOT NULL, default false	Czy zaakceptowane przez moderatora
`Lat`	double	NULL	Szerokość geogr.
`Lng`	double	NULL	Długość geogr.
`LocationLabel`	varchar	NULL	Opis lokalizacji
`Date`	varchar	NOT NULL	Data: `YYYY` / `YYYY-MM` / `YYYY-MM-DD`
`DatePrecision`	varchar	NOT NULL	`Year` / `Month` / `Day`
`Technique`	varchar	NULL	Technika fotograficzna
`InventoryNumber`	varchar	NULL	Nr inwentarzowy
`OriginalFormat`	varchar	NULL	Format oryginału
`License`	varchar	NULL	Licencja
`Digitization`	varchar	NULL	Parametry digitalizacji
`Quote`	text	NULL	Cytat / wspomnienie
`CreatedAt`	timestamp	NOT NULL	Data dodania (UTC)
`AuthorId`	int	FK → User	Autor materiału
`CategoryId`	int	FK → Cat.	Kategoria

Nowe zdjęcia trafiają do systemu z `IsApproved = false` i są widoczne wyłącznie w panelu moderacji admina (`/admin/moderacja`). Po akceptacji (`IsApproved = true`) zdjęcie pojawia się w publicznym archiwum i wynikach wyszukiwania. Publiczny endpo-

int `GET /api/photos` filtruje automatycznie tylko zatwierdzone materiały.

6.4 Tabela: Category

Tabela 8: Struktura tabeli Category

Kolumna	Typ	Ograniczenia	Opis
`Id`	int	PK, auto	Klucz główny
`Name`	varchar	NOT NULL	Nazwa kategorii
`Description`	text	NULL	Opis kategorii
`ParentId`	int	FK → Cat. RESTRICT	Kategoria nadrzędna

6.5 Tabela: Tag

Tabela 9: Struktura tabeli Tag

Kolumna	Typ	Ograniczenia	Opis
`Id`	int	PK, auto	Klucz główny
`Name`	varchar	NOT NULL, UNIQUE	Nazwa tagu

Relacja `Photo` ↔ `Tag` jest wielokrotna (M:N). Entity Framework Core automatycznie zarządza tabelą pośrednią `PhotoTag`.

6.6 Tabela: Comment

Tabela 10: Struktura tabeli Comment

Kolumna	Typ	Ograniczenia	Opis
`Id`	int	PK, auto	Klucz główny
`Text`	text	NOT NULL	Treść komentarza
`CreatedAt`	timestamp	NOT NULL	Data dodania (UTC)
`UpdatedAt`	timestamp	NULL	Data edycji (UTC)
`PhotoId`	int	FK → Photo, CASCADE	Powiązane zdjęcie
`AuthorId`	int	FK → User, CASCADE	Autor komentarza

7 Struktura metadanych

7.1 Klasyfikacja metadanych

Metadane zdjęcia podzielono na cztery grupy funkcjonalne:

7.1.1 Grupa 1: Identyfikacyjne (wymagane)

Tabela 11: Metadane identyfikacyjne

Pole	**Opis**	**Przykład**
`Title`	Tytuł zdjęcia	"Rynek Główny w Krakowie, 1965"
`CategoryId`	Przypisanie do hierarchii	Kraków > Stare Miasto
`Date`	Data wykonania zdjęcia	`1965`, `1965-06`
`DatePrecision`	Granularność daty	`Year / Month / Day`

7.1.2 Grupa 2: Lokalizacyjne

Tabela 12: Metadane lokalizacyjne

Pole	**Opis**	**Przykład**
`Lat`	Szerokość geograficzna	`50.0614`
`Lng`	Długość geograficzna	`19.9372`
`LocationLabel`	Czytelny opis miejsca	"Kraków, Rynek Główny"

7.1.3 Grupa 3: Opisowe

Tabela 13: Metadane opisowe

Pole	**Opis**	**Przykład**
`Description`	Rozbudowany opis tekstowy	"Widok na fontannę przed renowacją..."
`Quote`	Cytat, wspomnienie, anegdota	"Mój dziadek pracował w tej kamienicy..."
`Tags`	Swobodne słowa kluczowe	`architektura, fontanna, lata-60`

7.1.4 Grupa 4: Techniczne / archiwistyczne

Tabela 14: Metadane techniczne

Pole	**Opis**	**Przykład**
`Technique`	Technika fotograficzna	"Odbitka srebrowo-żelatynowa"
`OriginalFormat`	Format fizycznego oryginału	"13×18 cm", "35 mm klisza"
`InventoryNumber`	Numer w archiwum	"HA/2023/K-102"
`License`	Licencja praw autorskich	"CC BY-NC 4.0"
`Digitization`	Parametry digitalizacji	"600 DPI, TIFF 16-bit"

7.2 Uzasadnienie struktury metadanych

Zmienna precyzja daty jest kluczową decyzją projektową. Materiały historyczne rzadko posiadają dokładną datację. Przechowywanie daty jako stringa zamiast `DateTime` pozwala na:

- zapis niepełnej daty bez sztucznych wartości domyślnych (np. `1965-01-01`),
- inteligentne filtrowanie zakresów (zapytanie `dateTo=1970` zwróci wszystkie zdjęcia z lat 60.),
- czytelne wyświetlanie w UI zgodne z tym, co faktycznie wiadomo o zdjęciu.

Tagi swobodne uzupełniają sztywną hierarchię kategorii. Pozwalają na przekrojowe wyszukiwanie (np. wszystkie zdjęcia z tagiem `tramwaj` niezależnie od lokalizacji) i dają twórcom elastyczność w opisywaniu treści.

Metadane archiwistyczne (InventoryNumber, Technique, OriginalFormat, Digitization) są opcjonalne, ale niezbędne dla materiałów z formalnych kolekcji. Umożliwiają zachowanie ciągłości z papierową dokumentacją archiwów i bibliotek.

7.3 Parametry wyszukiwania a pola bazy danych

Tabela 15: Mapowanie parametrów wyszukiwania na pola bazy danych

Parametr API	Pola bazy danych	Typ dopasowania
`q`	`Title, Description, LocationLabel`	LIKE (contains)
`categoryId`	`CategoryId` + poddrzewo	IN (zbiór ID, BFS)
`tag`	`Tags.Name`	Równość (relacja M:N)
`dateFrom`	`Date`	Porównanie leksykograficzne
`dateTo`	`Date`	Porównanie leksykograficzne
`sortBy=date`	`Date`	Porządkowanie leksykograficzne
`sortBy=createdAt`	`CreatedAt`	Porządkowanie chronologiczne

8 Dokumentacja API

Baza URL: `http://localhost:5052/api` (środowisko development).

8.1 Legenda dostępu

Tabela 16: Poziomy dostępu do endpointów

Symbol	Znaczenie
[PUB]	Dostęp bez logowania — część publicznego API REST
[TWÓRCA]	Wymaga nagłówka `Authorization: Bearer <JWT>`
[ADMIN]	Wymaga JWT z claimem roli `admin`

8.2 Autentykacja (`/api/auth`)

Tabela 17: Endpointy autentykacji

Met.	Endpoint	Dostęp	Opis
POST	`/api/auth/google`	**[PUB]**	Logowanie przez Google OAuth. Body: `{ "credential": "..."}`
POST	`/api/auth/facebook`	**[PUB]**	Logowanie przez Facebook. Body: `{ "accessToken": "..."}`
GET	`/api/auth/me`	**[TWÓRCA]**	Dane zalogowanego użytkownika

8.3 Zdjęcia (`/api/photos`)

Tabela 18: Endpointy zarządzania zdjęciami

Met.	Endpoint	Dostęp	Opis
GET	`/api/photos`	**[PUB]**	Wyszukiwanie i paginacja zdjęć **zaakceptowanych** (`IsApproved = true`)
GET	`/api/photos/{id}`	**[PUB]**	Pełne metadane zdjęcia
GET	`/api/photos/my`	**[TWÓRCA]**	Lista własnych zdjęć
POST	`/api/photos`	**[TWÓRCA]**	Upload nowego zdjęcia (`multipart/form-data`)
PUT	`/api/photos/{id}`	**[TWÓRCA / ADMIN]**	Edycja metadanych (autor: własne; admin: dowolne)
DELETE	`/api/photos/{id}`	**[TWÓRCA / ADMIN]**	Usunięcie zdjęcia z dysku (autor: własne; admin: dowolne)

8.3.1 Parametry `GET /api/photos`

Tabela 19: Parametry wyszukiwania zdjęć

Parametr	Typ	Opis
`q`	string	Wyszukiwanie pełnotekstowe po `Title`, `Description`, `LocationLabel`
`categoryId`	int	Filtrowanie po kategorii — zwraca zdjęcia z wybranej kategorii **i wszystkich jej podkategorii** (BFS po drzewie)
`tag`	string	Filtrowanie po nazwie tagu (np. `?tag=architektura`)
`dateFrom`	string	Dolna granica daty: `YYYY`, `YYYY-MM` lub `YYYY-MM-DD`
`dateTo`	string	Górna granica daty (rok dopełniany do `YYYY-12-31`)
`sortBy`	string	`date` lub `createdAt` (domyślnie: `createdAt`)
`sortDir`	string	`asc` lub `desc` (domyślnie: `desc`)
`page`	int	Numer strony (domyślnie: 1)
`pageSize`	int	Wyników na stronę (domyślnie: 12)

8.3.2 Body `POST /api/photos (multipart/form-data)`

Tabela 20: Pola formularza uploadu zdjęcia

Pole	Wymagane	Opis
`file`	tak	Plik obrazu (jpg, png, webp itp.)
`title`	tak	Tytuł zdjęcia (max 200 znaków)
`categoryId`	tak	ID kategorii hierarchicznej
`date`	tak	Data: `YYYY`, `YYYY-MM` lub `YYYY-MM-DD`
`description`	nie	Opis tekstowy (max 2000 znaków)
`lat`	nie	Szerokość geograficzna
`lng`	nie	Długość geograficzna
`locationLabel`	nie	Czytelny opis lokalizacji
`technique`	nie	Technika fotograficzna
`quote`	nie	Cytat / wspomnienie (max 1000 znaków)
`inventoryNumber`	nie	Numer inwentarzowy
`originalFormat`	nie	Format oryginału
`license`	nie	Licencja
`digitization`	nie	Informacje o digitalizacji
`tags`	nie	Tagi rozdzielone przecinkami

8.3.3 Body `PUT /api/photos/{id} (application/json)`

Wszystkie pola są opcjonalne — przesyłane są wyłącznie zmieniające się wartości:

Listing 7: Przykładowe ciało żądania PUT /api/photos/{id}

```
{
  "title":            "Nowy tytul",
  "description":      "Opis...",
  "categoryId":       3,
  "date":             "1965-06",
  "lat":              50.0614,
  "lng":              19.9372,
  "locationLabel":    "Krakow, Rynek Glowny",
  "technique":        "Sepia",
  "quote":            "Wspomnienie...",
  "inventoryNumber": "HA/2023/K-102",
  "originalFormat":  "13x18 cm",
  "license":          "CC BY-NC 4.0",
  "digitization":    "600 DPI",
  "tags":             "architektura, lata60"
}
```

8.4 Kategorie (/api/categories)

Tabela 21: Endpointy zarządzania kategoriami

Met.	Endpoint	Dostęp	Opis
GET	/api/categories	**[PUB]**	Płaska lista wszystkich kategorii
GET	/api/categories/{id}	**[PUB]**	Szczegóły kategorii
POST	/api/categories	**[ADMIN]**	Utworzenie kategorii
PUT	/api/categories/{id}	**[ADMIN]**	Edycja kategorii
DELETE	/api/categories/{id}	**[ADMIN]**	Usunięcie (blokowane przez RESTRICT)

8.5 Komentarze

Tabela 22: Endpointy komentarzy

Met.	Endpoint	Dostęp	Opis
GET	/api/photos/{photoId}/comments	**[PUB]**	Lista komentarzy do zdjęcia
POST	/api/photos/{photoId}/comments	**[TWÓRCA]**	Dodanie komentarza
PUT	/api/comments/{id}	**[TWÓRCA / ADMIN]**	Edycja (autor: własne; admin: dowolne)
DELETE	/api/comments/{id}	**[TWÓRCA / ADMIN]**	Usunięcie (autor: własne; admin: dowolne)

8.6 Administracja (/api/admin)

Wszystkie endpointy tej grupy wymagają roli `admin`.

Tabela 23: Endpointy panelu administracyjnego

Met.	Endpoint	Dostęp	Opis
GET	`/api/admin/users`	**[ADMIN]**	Paginowana lista użytkowników (`page`, `pageSize`)
PUT	`/api/admin/users/{id}/block`	**[ADMIN]**	Blokada użytkownika. Body: `{ "reason": "..."}`
PUT	`/api/admin/users/{id}/unblock`	**[ADMIN]**	Odblokowanie użytkownika
GET	`/api/admin/stats`	**[ADMIN]**	Statystyki: `totalPhotos`, `totalUsers`, `blockedUsers`, `pendingPhotos`
GET	`/api/admin/photos/pending`	**[ADMIN]**	Paginowana lista zdjęć oczekujących na moderację (`IsApproved = false`)
PUT	`/api/admin/photos/{id}/approve`	**[ADMIN]**	Akceptacja zdjęcia — ustawia `IsApproved = true`

8.7 Przykładowe odpowiedzi

8.7.1 `GET /api/photos` — odpowiedź stronicowana

Listing 8: Odpowiedź GET /api/photos

```
{
  "items": [
    {
      "id": 42,
      "title": "Rynek Glowny, 1965",
      "description": "Widok na fontanne z Neptunem...",
      "imagePath": "/uploads/abc123.jpg",
      "thumbnailPath": "/uploads/abc123_thumb.jpg",
      "lat": 50.0614,
      "lng": 19.9372,
      "locationLabel": "Krakow, Rynek Glowny",
      "date": "1965",
      "datePrecision": "year",
      "technique": null,
      "inventoryNumber": null,
      "quote": null,
      "createdAt": "2026-03-25T13:32:00Z",
      "author": { "id": 5, "displayName": "Jan Kowalski" },
      "category": { "id": 3, "name": "Stare Miasto" },
      "tags": [{ "id": 1, "name": "architektura" }]
    }
  ],
  "page": 1,
  "pageSize": 12,
```

```
  "totalCount": 87,
  "totalPages": 8
}
```

8.7.2 *POST /api/auth/google — odpowiedź po zalogowaniu*

Listing 9: Odpowiedź POST /api/auth/google

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 5,
    "displayName": "Jan Kowalski",
    "avatarUrl": "https://lh3.googleusercontent.com/...",
    "role": "creator",
    "hasGoogle": true,
    "hasFacebook": false
  }
}
```

Pola `hasGoogle` i `hasFacebook` informują frontend, przez jakich dostawców użytkownik ma powiązane konto, nie ujawniając surowych identyfikatorów OAuth.

8.8 Autoryzacja — szczegóły tokenu JWT

Token generowany algorytmem HMAC-SHA256, ważny przez 30 dni. Payload:

Listing 10: Payload tokenu JWT

```
{
  "nameid":      "5",          // userId
  "unique_name": "Jan Kowalski",
  "role":        "creator",    // viewer | creator | admin
  "exp":         1753000000
}
```

Przekazywany w nagłówku każdego chronionego żądania:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

8.9 Zestawienie wszystkich endpointów

Tabela 24: Kompletna tabela endpointów z poziomem dostępu

Met.	Endpoint	Pub.	Twórca	Admin
POST	`/api/auth/google`	✓		
POST	`/api/auth/facebook`	✓		
GET	`/api/auth/me`		✓	
GET	`/api/photos`	✓		
GET	`/api/photos/{id}`	✓		

Tabela 24 — ciąg dalszy

Met.	Endpoint	Pub.	Twórca	Admin
GET	/api/photos/my		✓	
POST	/api/photos		✓	
PUT	/api/photos/{id}		✓(własne)	✓(wszystkie)
DELETE	/api/photos/{id}		✓(własne)	✓(wszystkie)
GET	/api/categories	✓		
GET	/api/categories/{id}	✓		
POST	/api/categories			✓
PUT	/api/categories/{id}			✓
DELETE	/api/categories/{id}			✓
GET	/api/photos/{photoId}/comments	✓		
POST	/api/photos/{photoId}/comments		✓	
PUT	/api/comments/{id}		✓(własne)	✓(wszystkie)
DELETE	/api/comments/{id}		✓(własne)	✓(wszystkie)
GET	/api/admin/users			✓
PUT	/api/admin/users/{id}/block			✓
PUT	/api/admin/users/{id}/unblock			✓
GET	/api/admin/stats			✓
GET	/api/admin/photos/pending			✓
PUT	/api/admin/photos/{id}/approve			✓

Łącznie: 24 endpointy — 7 publicznych, 6 wyłącznie dla twórcy, 11 wyłącznie dla admina.

9 Analiza dostępności informacji (WCAG 2.1 AA)

9.1 Przegląd

Standard WCAG 2.1 poziom AA obejmuje cztery zasady: **Postrzegalność**, **Funkcjonalność**, **Zrozumiałość** i **Solidność** (POUR).

9.2 Postrzegalność (Perceivable)

9.2.1 1.1.1 — Treść nietekstowa (poziom A)

Wszystkie zdjęcia posiadają obowiązkowy atrybut `alt` ustawiony na tytuł zdjęcia. Ikony dekoracyjne oznaczone są `aria-hidden="true"`.

Listing 11: Przykład atrybutu alt

```
<img :src="photo.thumbnailUrl" :alt="photo.title" />
<span class="material-symbols-outlined" aria-hidden="true">
   delete</span>
```

9.2.2 1.3.1 — Informacje i relacje (poziom A)

Formularze używają jawnych elementów `<label for="...»` powiązanych z polami przez atrybut `id`. Strona używa semantycznego HTML5: `<nav>`, `<main>`, `<header>`,

`<footer>`, `<aside>`, `<section>`.

Listing 12: Jawne etykiety formularza

```
<label for="pu-title">Tytul zdjecia *</label>
<input id="pu-title" v-model="form.title" type="text" required /
   >
```

9.2.3 1.4.3 — Kontrast (poziom AA)

Tabela 25: Kontrast kolorów w poszczególnych motywach

Motyw	Tekst	Tło	Wsp. kontrastu	Status
`light`	`#1c1c17`	`#fcf9f1`	≈17:1	✓
`dark`	`#e8e5d8`	`#1a1a14`	≈14:1	✓
`high-contrast`	`#00ffff`	`#000000`	21:1	✓

9.2.4 1.4.4 — Zmiana rozmiaru tekstu (poziom AA)

System udostępnia trzy rozmiary czcionki: `small`, `normal`, `large` (przełączane przez `SettingsDropdown`). Typografia zbudowana jest na CSS `clamp()` i zmiennych CSS. Wybór zapisywany w `localStorage` (klucz `cas_fontsize`).

9.2.5 1.4.13 — Zawartość przy najechaniu / fokusie (poziom AA)

Powiadomienia toast (`AppToast`) wyświetlają się czasowo i nie blokują interakcji. Nie są stosowane niestandardowe tooltip-y mogące przesłaniać treść.

9.3 Funkcjonalność (Operable)

9.3.1 2.1.1 — Klawiatura (poziom A)

Wszystkie interaktywne elementy są dostępne przez klawiaturę:

- fokus klawiatury widoczny przez `box-shadow: var(-focus-ring)` na wszystkich elementach `:focus-visible`,
- menu mobilne zarządza `tabindex` elementów wewnątrz (`-1` gdy zamknięte, `0` gdy otwarte),
- dialogi (`BlockUserDialog`) ustawiają `aria-modal="true"`.

9.3.2 2.4.1 — Pomijanie bloków (poziom A)

Skip-link „Przejdź do treści"dostępny jako pierwszy element na stronie. Widoczny tylko przy fokusie klawiatury:

Listing 13: Implementacja skip-linka

```
<!-- App.vue -->
<a href="#main-content" class="skip-link">Przejdz do tresci</a>
```

```
/* CSS */
.skip-link {
  position: absolute;
  top: -100%;
}
.skip-link:focus {
  top: 8px;
}
```

9.3.3 2.4.4 — Cel linku (poziom A)

Linki paginacji mają `aria-label` z numerem strony. Linki nawigacji mają `aria-current="page"` na aktywnej pozycji.

9.3.4 2.4.6 — Nagłówki i etykiety (poziom AA)

Każda strona posiada jeden nagłówek `<h1>`. Hierarchia `h1` → `h2` → `h3` jest zachowana. Sekcje filtrów i wyników posiadają nagłówki.

9.3.5 2.3.3 — Animacja z interakcji (poziom AAA — zrealizowane)

Reguła CSS `@media (prefers-reduced-motion: reduce)` wyłącza animacje przejść między stronami i `scroll-behavior: smooth`.

9.4 Zrozumiałość (Understandable)

9.4.1 3.1.1 — Język strony (poziom A)

Atrybut `lang="pl"` na elemencie `<html>`. Cały interfejs jest w języku polskim, obsługiwany przez Vue i18n.

9.4.2 3.3.1 — Identyfikacja błędu (poziom A)

Błędy walidacji wyświetlane w elemencie `<p role="alert»` bezpośrednio pod polem. Toast z komunikatem błędu posiada `aria-live="assertive"`.

9.4.3 3.3.2 — Etykiety lub instrukcje (poziom A)

Każde pole formularza posiada widoczną etykietę. Pola opcjonalne i wymagane oznaczone są gwiazdką `*` z wyjaśnieniem w kontekście formularza. Pole tagów zawiera wskazówkę „Oddziel tagi przecinkami".

9.5 Solidność (Robust)

9.5.1 4.1.2 — Nazwa, rola, wartość (poziom A)

Komponenty interaktywne używają natywnych elementów HTML. Przyciski przełączające używają `aria-pressed`:

Listing 14: Użycie aria-pressed na chipach filtrów

```
<button :aria-pressed="categoryId␣===␣cat.id">
  {{ cat.name }}
</button>
```

9.5.2 4.1.3 — Komunikaty o stanie (poziom AA)

Composable `useAnnouncer` zarządza regionem ARIA live. Zmiany liczby wyników, wysyłka formularzy i komunikaty o błędach są ogłaszane przez czytniki ekranu bez przenoszenia fokusu:

Listing 15: Region ARIA live w App.vue

```
<div class="sr-only"
     :aria-live="politeness"
     aria-atomic="true"
     role="status">
  {{ announcement }}
</div>
```

9.6 Zestawienie kryteriów WCAG 2.1 AA

Tabela 26: Status realizacji kryteriów WCAG 2.1 AA

Kryterium	Opis	Status
1.1.1	Treść nietekstowa (alt)	✓
1.3.1	Informacje i relacje (semantyka HTML, label)	✓
1.3.4	Orientacja (brak blokady)	✓
1.4.1	Użycie koloru (nie jedyny nośnik informacji)	✓
1.4.3	Kontrast tekstu (trzy motywy)	✓
1.4.4	Zmiana rozmiaru tekstu (trzy skale)	✓
1.4.10	Przepływ tekstu (responsywny layout)	✓
1.4.11	Kontrast elementów nietekstowych	✓
1.4.12	Odstępy w tekście (brak blokady CSS)	✓
2.1.1	Klawiatura	✓
2.1.2	Brak pułapki klawiaturowej	✓
2.3.3	Animacja z interakcji (`prefers-reduced-motion`)	✓
2.4.1	Pomijanie bloków (skip-link)	✓
2.4.3	Kolejność fokusu (logiczna kolejność DOM)	✓
2.4.4	Cel linku	✓
2.4.6	Nagłówki i etykiety	✓
2.4.7	Widoczny fokus (focus-ring)	✓
3.1.1	Język strony (`lang="pl"`)	✓
3.2.1	Po fokusie (brak auto zmian kontekstu)	✓

Tabela 26 — ciąg dalszy

Kryterium	Opis	Status
3.3.1	Identyfikacja błędu (`role="alert"`)	✓
3.3.2	Etykiety lub instrukcje	✓
4.1.1	Parsowanie (poprawny HTML)	✓
4.1.2	Nazwa, rola, wartość (natywne elementy + ARIA)	✓
4.1.3	Komunikaty o stanie (`useAnnouncer`)	✓

9.7 Dostosowanie wyglądu

System implementuje pełną trójkę wariantów wyglądu:

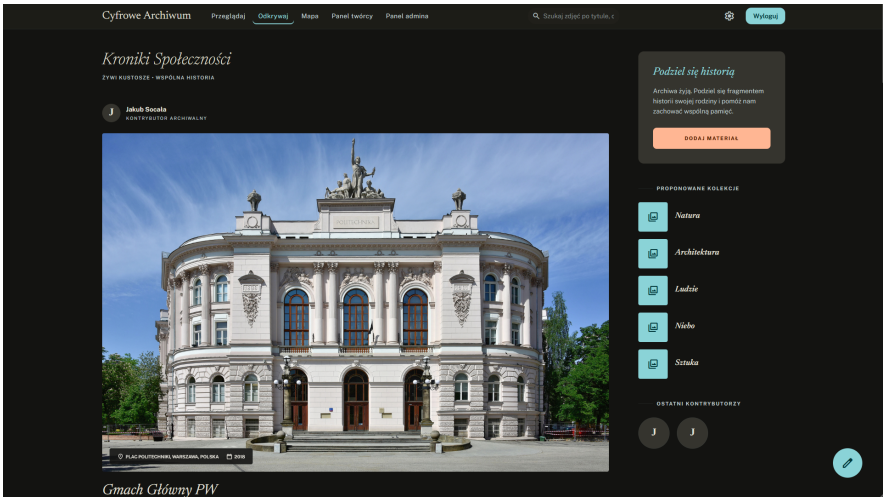
Tabela 27: Motywy wizualne i skale czcionki

Motyw	Trigger	Przeznaczenie
`light`	Domyślny lub `prefers-color-scheme: light`	Standardowe użytkowanie
`dark`	`prefers-color-scheme: dark` lub ręczny	Komfort w słabym oświetleniu
`high-contrast`	`prefers-contrast: more` lub ręczny	Osoby słabowidzące

Wybór motywu zapisywany w `localStorage` (klucz `cas_theme`). Zmiana następuje natychmiastowo przez atrybut `data-theme` na `<html>` bez przeładowania strony.

Skala czcionki zapisywana osobno (klucz `cas_fontsize`):

- `small` — baza 0.875 rem,
- `normal` — baza 1 rem,
- `large` — baza 1.125 rem.



Rysunek 5: Aplikacja po wybraniu motywu kontrastowego